Dictionary Compression in Information Retrieval: Analysis of Prefix Front Coding and Varints

Naufal Bagas Ganefriady
Department of Computer Science and Communications Engineering
Waseda University, Tokyo, Japan
1W25CF88 - naufalbagasg@waseda.akane.jp

Abstract

In contemporary information retrieval systems, the processing of user queries requires the efficient mapping of textual terms to their corresponding numeric identifiers within a compressed dictionary. As datasets grow to billions of documents, the memory used by these dictionaries becomes a critical bottleneck. This paper investigates dictionary compression techniques with a focus on Prefix Front Coding (PFC), a method that exploits the fact that alphabetically sorted words often share the same starting characters. Through algorithmic analysis and comparison, it show that the perceived trade off between *compression ratio* and *access speed* can be greatly reduced through intelligent block management and statistical sampling. The findings provide practical guidance for engineers and researchers working on memory-efficient IR and database systems.

Keywords: dictionary compression; prefix front coding; Re-Pair; information retrieval; string dictionaries; block compression; grammar-based compression; inverted index

1 Research Context and Motivation

This paper grew out of coursework in Information Retrieval at Waseda University, where a discussion on dictionary compression techniques sparked a deeper investigation into one particular method: Prefix Front Coding. While the initial discussion only touched on front coding briefly, the underlying question felt important enough to explore further, how do systems like Google, Spotify, or SAP actually store and search through hundreds of millions of words efficiently? The answer, as it turns out, involves a surprisingly elegant chain of ideas that this paper aims to unpack.

The goal is not just to describe the algorithms whether to understand *why* each one was designed the way it was and what real world problem it was trying to solve.

2 Introduction

Building a search engine for a major website with 500 million product listings present every product title and description is broken into individual words, and system must instantly answer the question: *"Does this word exist, and if so, which documents contain it?"* The data structure that answers this question is called a **dictionary** which a mapping from words (terms) to their numeric identifiers (IDs) then used to look up posting lists in an inverted index.

At first glance, a simple sorted list or a hash table seems sufficient. But when the vocabulary reaches tens or hundreds of millions of distinct terms, the dictionary itself becomes a memory

problem. A vocabulary of 50 million terms with an average word length of 12 characters consumes around 600 MB of RAM in a naive representation. For a production server that must simultaneously run thousands of queries per second, dedicating that much memory to the dictionary alone is expensive and if it no longer fits in RAM, every single lookup triggers a slow disk access.

Dictionary compression solves this by making the stored representation of terms much smaller, while still allowing two fundamental operations to happen quickly:

- **Locate**: Given a word (e.g., `"compression"`), find its numeric ID as fast as possible.
- **Extract**: Given a numeric ID (e.g., 4827), retrieve the original word.

Both operations must complete within microseconds (μs) for the system to remain responsive. This paper examines how Prefix Front Coding (PFC) achieves this, and how its successors improve the compression ratio even further without sacrificing too much speed.

2.1 Why This Problem Is Still Relevant Today

Even with modern hardware, dictionary compression remains an active topic. Cloud databases like SAP HANA explicitly use front coding because the compression is "free", it requires no heavy computation, yet reduces memory usage significantly. In the era of large language models and semantic search, the vocabulary explosion has only accelerated: systems now index subword tokens, URIs, biological sequences, and emoji strings in addition to natural-language words.

3 Background: How Inverted Indexes Work

An inverted index stores, for each unique term t, a **posting list**: the list of document IDs that contain t.

Term	Posting List
`compress`	$[4, 19, 203, 1041, \ldots]$
`compressed`	$[4, 45, 203, \ldots]$
`compressor`	$[19, 500, \ldots]$

The dictionary is the structure that maps each term to the location of its posting list. Notice that in the example above, three terms share the same 8-character prefix `compress`. Prefix Front Coding exploits exactly this observation.

4 Prefix Front Coding (PFC)

4.1 Core Idea

PFC is a differential compression technique: instead of storing each term in full, it stores only the *difference* between each term and the one before it in sorted order.

4.2 Bucket Structure

Terms are grouped into small clusters called **buckets**, each containing k terms (e.g., $k = 16$):

1. The first term in each bucket (*bucket header*) is stored in full.
2. Every other term is encoded as a pair: (length of shared prefix with previous term, remaining suffix).

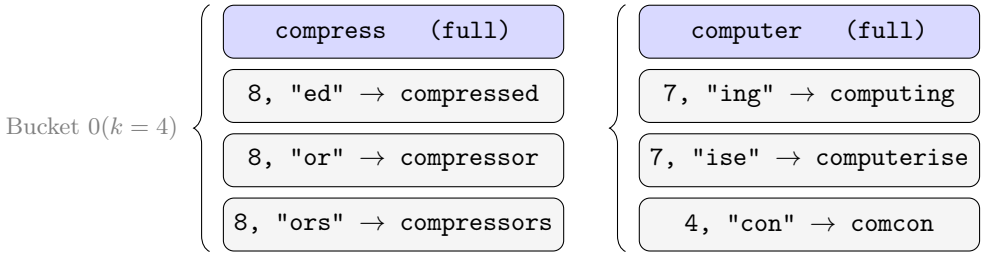

Figure 1: A PFC bucket example with $k = 4$.

4.3 Reconstruction Formula

$$t_i = \underbrace{t_{i-1}[1\,..\,\ell_i]}_{\text{shared prefix}} \parallel \underbrace{s_i}_{\text{new suffix}} \tag{1}$$

5 Re-Pair Front Coding (RPFC)

PFC only exploits *prefix* similarity. Re-Pair Front Coding (RPFC) targets recurring suffixes using the Re-Pair grammar-based compression algorithm.

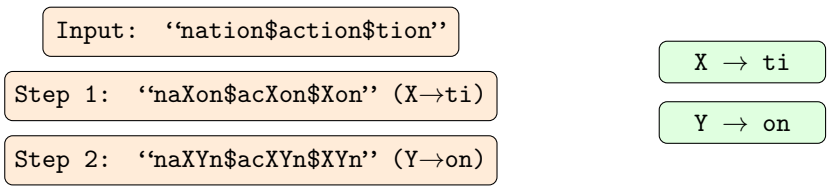

Figure 2: Re-Pair substitution example.

6 Block-Based RPFC (BRPFC) and the Sampled Variant

BRPFC splits the dictionary into small independent blocks (e.g., 64 KB). This allows for:

- **Fine-grained access**: Only one block needs decompression for a lookup.
- **Parallelism**: Blocks can be processed across multiple CPU cores.

Sampled BRPFC trains a shared grammar on a 2–5% sample of all suffixes, reducing construction time drastically.

7 Comparative Analysis

Table 1: Comparison of dictionary compression schemes.

Scheme	**CR**	**Latency**	**Memory Use**	**Parallel**
Plain sorted	Low	Very fast	Low	Yes
PFC	Medium	Fast	Low	Partial
Global RPFC	High	Moderate	Very high	No
BRPFC	High	Fast	Low	Yes

8 Conclusion

The evolution from PFC to Sampled BRPFC demonstrates that exploiting data structure, partitioning memory, and using statistical approximation can yield systems that are simultaneously more compact and faster. These combined optimisations bring grammar-based front coding into a practical range for production database systems.

Acknowledgements

The author thanks the faculty of the Department of Computer Science and Communications Engineering at Waseda University for stimulating discussions on information retrieval and data compression during the 2025 academic year.

References

[1] Manning, C.D.; Raghavan, P.; Schütze, H. *Introduction to Information Retrieval*; 2008.

[2] Lasch, R.; et al. Faster & strong: string dictionary compression using sampling and fast vectorized decompression. *The VLDB Journal* 2020.